CodInClub
Powered by BridgeLabz

Day 2 Live-Coding Session — 5 Problems with Solutions

1. Vowel & Consonant Counter

Scenario: A library orientation kiosk counts vowels and consonants in a submitted book title for a simple text-stats display.

Task:

• Accept a string (assume only letters and spaces).
• Loop through each character using charAt().
• Count vowels (a, e, i, o, u — case-insensitive) and consonants separately; ignore spaces.
• Print the total vowels and total consonants.

Suggested method signature(s): `void countVowelsAndConsonants(String text)`

Sample Input / Output:

Input	Output
"Java Programming"	Vowels: 5 | Consonants: 10

Concepts covered: charAt(), length(), loops, case-insensitive character comparison.

Java Solution:

```
public static void countVowelsAndConsonants(String text) {
    int vowels = 0, consonants = 0;
    String lower = text.toLowerCase();
    for (int i = 0; i < lower.length(); i++) {
        char ch = lower.charAt(i);
        if (ch >= 'a' && ch <= 'z') {
            if (ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o' || ch == 'u') {
                vowels++;
            } else {
                consonants++;
            }
        }
    }
    System.out.println("Vowels: " + vowels + " | Consonants: " + consonants);
}
```

2. CSV Student Record Parser

Scenario: The T&P; team receives student registration data as CSV lines and needs a quick parser to split each line into fields and print a formatted record.

Task:

• Accept a CSV line in the form "Name,RollNumber,Department".
• Use split(",") to break it into fields.
• Validate that exactly 3 fields are present; if not, print "Invalid Record".
• Print a formatted record: "Name: ... | Roll No: ... | Dept: ...".

Suggested method signature(s): `void parseStudentRecord(String csvLine)`

Sample Input / Output:

Input	Output
"Ananya Verma,RA2211003010123,CSE"	Name: Ananya Verma | Roll No: RA2211003010123 | Dept: CSE
"Ananya Verma,CSE"	Invalid Record

Concepts covered: split(), array length validation, string concatenation, formatted output.

Java Solution:

```
public static void parseStudentRecord(String csvLine) {
    String[] fields = csvLine.split(",");
    if (fields.length != 3) {
        System.out.println("Invalid Record");
        return;
    }
    String name = fields[0].trim();
    String rollNo = fields[1].trim();
    String dept = fields[2].trim();
    System.out.println("Name: " + name + " | Roll No: " + rollNo + " | Dept: " + dept);
}
```

3. File Extension Validator

Scenario: An assignment-upload portal must check whether an uploaded filename has an accepted extension (pdf, docx, zip) regardless of case, before accepting the submission.

Task:

• Accept a filename string.
• Find the last '.' using lastIndexOf('.') and extract the extension with substring().
• Compare the extension case-insensitively against the accepted list: pdf, docx, zip.
• Print "Accepted" or "Rejected — invalid file type".

Suggested method signature(s): `String validateFileExtension(String filename)`

Sample Input / Output:

Input	Output
"Assignment1.PDF"	Accepted
"notes.txt"	Rejected — invalid file type

Concepts covered: lastIndexOf(), substring(), equalsIgnoreCase(), conditional logic.

Java Solution:

```
public static String validateFileExtension(String filename) {
    int lastDot = filename.lastIndexOf('.');
    if (lastDot == -1 || lastDot == filename.length() - 1) {
        return "Rejected – invalid file type";
    }
    String ext = filename.substring(lastDot + 1);
    if (ext.equalsIgnoreCase("pdf") || ext.equalsIgnoreCase("docx") || ext.equalsIgnoreCase("zi
p")) {
        return "Accepted";
    } else {
        return "Rejected – invalid file type";
    }
}
```

4. Masked Phone Number Formatter

Scenario: A student-support call center displays a partially masked version of a registered phone number on-screen for privacy, while agents confirm identity using the last 4 digits.

Task:

• Accept a phone number as a string.
• Validate that it is exactly 10 digits (all numeric).
• Build a masked version showing "XXXXXX" followed by the last 4 digits, using StringBuilder.
• Insert a "-" between the mask and the last 4 digits for readability.
• Print the final masked number, or an error message if validation fails.

Suggested method signature(s): `String maskPhoneNumber(String phone)`

Sample Input / Output:

Input	Output
"9876543210"	XXXXXX-3210
"98765"	Invalid phone number

Concepts covered: String length/digit validation, substring(), StringBuilder insert(), masking patterns.

Java Solution:

```
public static String maskPhoneNumber(String phone) {
    if (phone == null || phone.length() != 10) {
        return "Invalid phone number";
    }
    for (int i = 0; i < phone.length(); i++) {
        if (!Character.isDigit(phone.charAt(i))) {
            return "Invalid phone number";
        }
    }
    String last4 = phone.substring(6);
    StringBuilder sb = new StringBuilder("XXXXXX");
    sb.insert(6, "-");
    sb.append(last4);
    return sb.toString();
}
```

5. Bank Transaction Reference Generator & Validator

Scenario: A fintech onboarding module for a placement-prep hackathon needs to both normalize and validate transaction reference codes. A valid reference is exactly 14 characters: 3 letters (bank code) + 6 digits (date, ddMMyy) + 5 digits (sequence number). Users sometimes paste codes with stray spaces or a mixed-case bank code.

Task:

• Accept a raw reference string that may contain leading/trailing spaces.
• Normalize it: trim() the spaces, then uppercase only the first 3 characters using substring() + concatenation — leave the rest untouched.
• Validate: exactly 14 characters after normalization; the first 3 characters are letters; the remaining 11 are digits (use Character.isLetter() / isDigit() in a loop — no regex).
• If valid, build a formatted display line with StringBuilder: "[BANKCODE] DATE: dd/MM/yy | SEQ: 12345".
• If invalid, print the specific reason: wrong length, non-letter bank code, or non-digit body.

Suggested method signature(s): `String normalizeReference(String raw) + String validateAndFormat(String reference)`

Sample Input / Output:

Input	Output
" hdf03022600042 "	[HDF] DATE: 03/02/26 | SEQ: 00042
"12F03022600042"	Invalid: bank code must be 3 letters

Concepts covered: trim(), substring(), string concatenation, Character.isLetter()/isDigit(), StringBuilder, multi-stage validation.

Java Solution:

```
public static String normalizeReference(String raw) {
    if (raw == null) return "";
    String trimmed = raw.trim();
    if (trimmed.length() < 3) return trimmed.toUpperCase();
    String bankCode = trimmed.substring(0, 3).toUpperCase();
    String rest = trimmed.substring(3);
    return bankCode + rest;
}

public static String validateAndFormat(String reference) {
    if (reference.length() != 14) {
        return "Invalid: wrong length";
    }
    for (int i = 0; i < 3; i++) {
        if (!Character.isLetter(reference.charAt(i))) {
            return "Invalid: bank code must be 3 letters";
        }
    }
    for (int i = 3; i < 14; i++) {
        if (!Character.isDigit(reference.charAt(i))) {
            return "Invalid: non-digit body";
        }
    }
    String bank = reference.substring(0, 3);
    String dd = reference.substring(3, 5);
    String mm = reference.substring(5, 7);
    String yy = reference.substring(7, 9);
    String seq = reference.substring(9, 14);

    StringBuilder sb = new StringBuilder();
    sb.append("[").append(bank).append("] DATE: ")
      .append(dd).append("/").append(mm).append("/").append(yy)
      .append(" | SEQ: ").append(seq);
    return sb.toString();
}
```